

MIGUEL ANGEL SALAZAR MARTINEZ

Universidad Politécnica de San Luis Potosi

CNO 4 – Seguridad Informática

Profesor: Servando López Contreras

Actividad 11 – File Path Traversal | Validation of
file extension with null byte bypass

25/09/26

Introducción

En el ámbito de la ciberseguridad, las defensas más comunes suelen basarse en la validación superficial de la entrada: comprobar que un archivo tenga cierta extensión, que la ruta empiece por un directorio concreto, o que no contenga secuencias sospechosas. Sin embargo, estas comprobaciones a nivel de cadena pueden ser engañadas cuando el atacante explota discrepancias entre cómo la aplicación interpreta la entrada y cómo la interpreta el sistema subyacente. El caso del Null Byte Injection (inyección de byte nulo) es un ejemplo clásico y peligroso de este tipo de fallo: la aplicación cree estar validando una ruta con extensión .png, pero el sistema de archivos, al encontrar un carácter nulo (%00), trunca la cadena y accede a un archivo completamente distinto. Comprender este mecanismo es esencial para entender por qué la validación debe realizarse después de la canonicalización completa y por qué los entornos antiguos representan un riesgo significativo incluso cuando el código "parece" seguro.

Marco Teórico

El byte nulo (%00 en URL encoding, o \0 en representación binaria) es un carácter especial que, en muchos lenguajes de programación de bajo nivel y en el sistema operativo, se utiliza como terminador de cadena. Esto significa que, cuando una función del sistema de archivos (escrita originalmente en C) recibe una cadena con un byte nulo, deja de leer en ese punto, ignorando todo lo que venga después.

En aplicaciones web escritas en lenguajes como PHP (versiones antiguas), Java o ASP, cuando la entrada del usuario se pasa directamente a funciones del sistema de archivos, ese byte nulo puede truncar la ruta en el punto donde el atacante lo inserta.

El mecanismo del ataque en este laboratorio

La aplicación de este laboratorio valida que el archivo solicitado tenga una extensión permitida (como .png). La comprobación se realiza a nivel de cadena: si el nombre termina en .png, se considera una imagen válida y se permite el acceso.

`.././../etc/passwd%00.png`

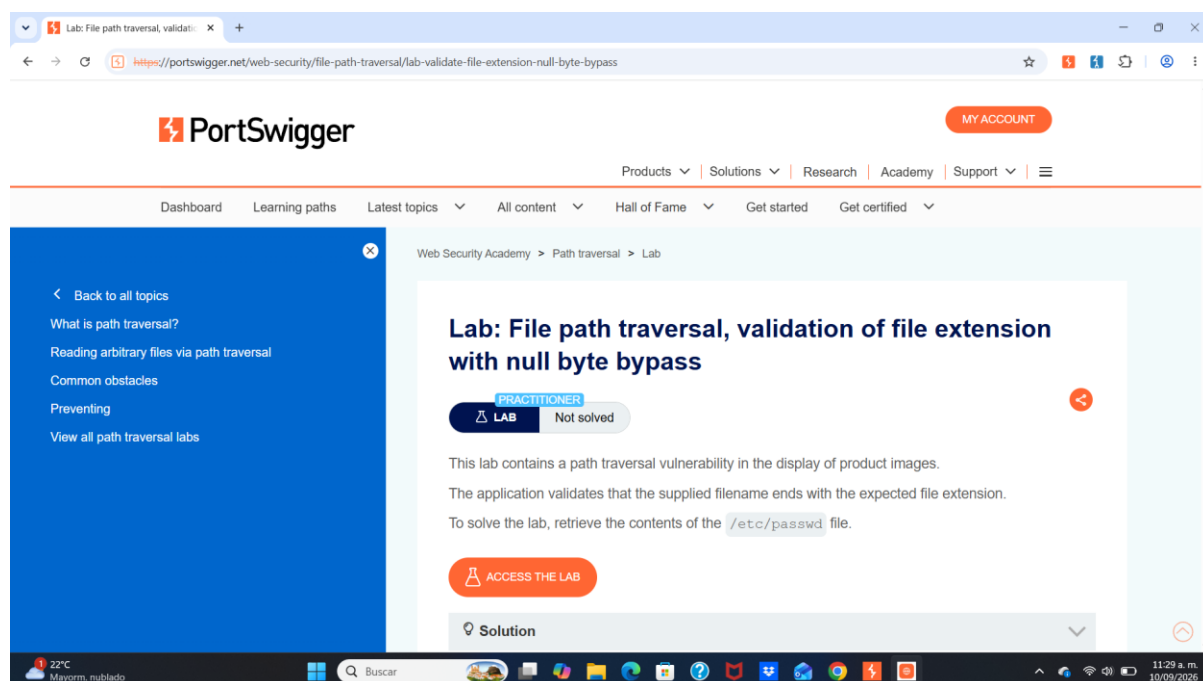
Cuando la aplicación verifica la extensión, ve que la cadena termina en .png y la validación pasa. Sin embargo, cuando la ruta se pasa a la función de apertura de archivos, el byte nulo %00 actúa como terminador: la función solo ve `.././../etc/passwd` y abre ese archivo. La extensión .png que la aplicación validó nunca llega a usarse.

Posibles soluciones

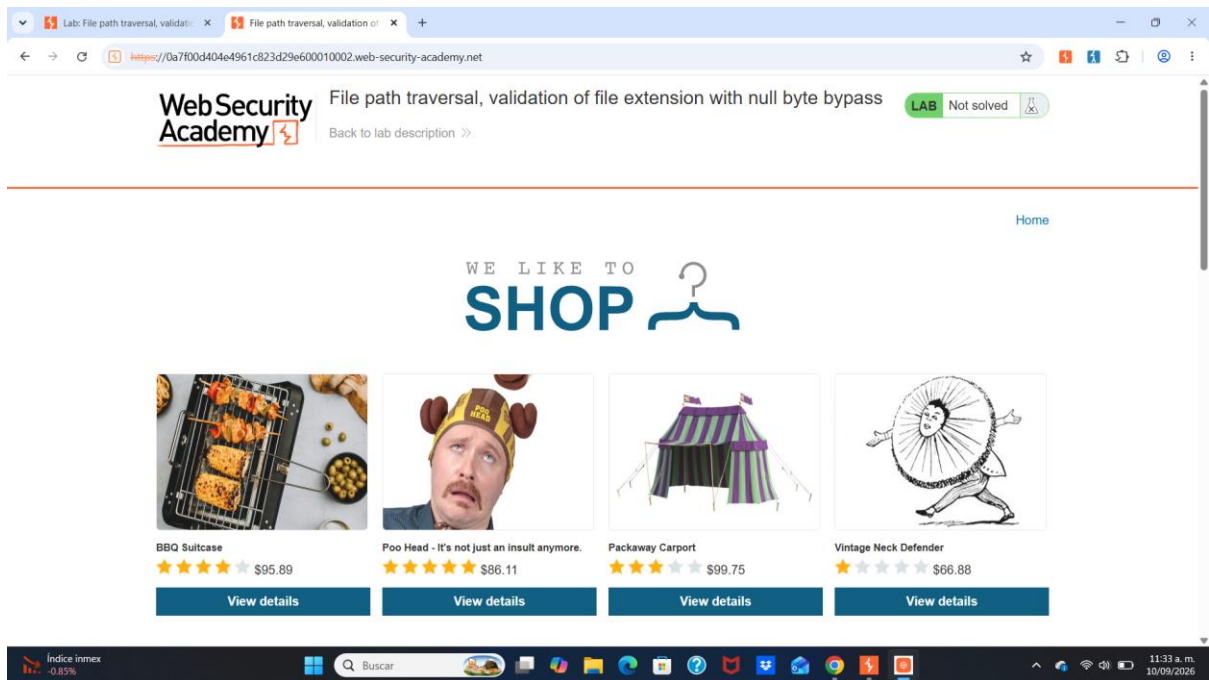
Soluciones para prevenir el Path Traversal con bypass de byte nulo

Para evitar esta vulnerabilidad, la defensa no debe basarse únicamente en la validación de la extensión del archivo a nivel de cadena, ya que un atacante puede usar el byte nulo (%00) para truncar la ruta antes de la extensión verificada. La solución correcta consiste en rechazar cualquier entrada que contenga bytes nulos antes de cualquier procesamiento, y canonicalizar la ruta completa (con `realpath()` o equivalente) verificando que el resultado final permanezca dentro del directorio permitido. Además, se debe usar una lista blanca de archivos permitidos e identificadores numéricos en lugar de nombres de archivo, y garantizar que el entorno de ejecución (PHP, Java, etc.) esté actualizado, ya que las versiones modernas rechazan los bytes nulos en rutas de archivo.

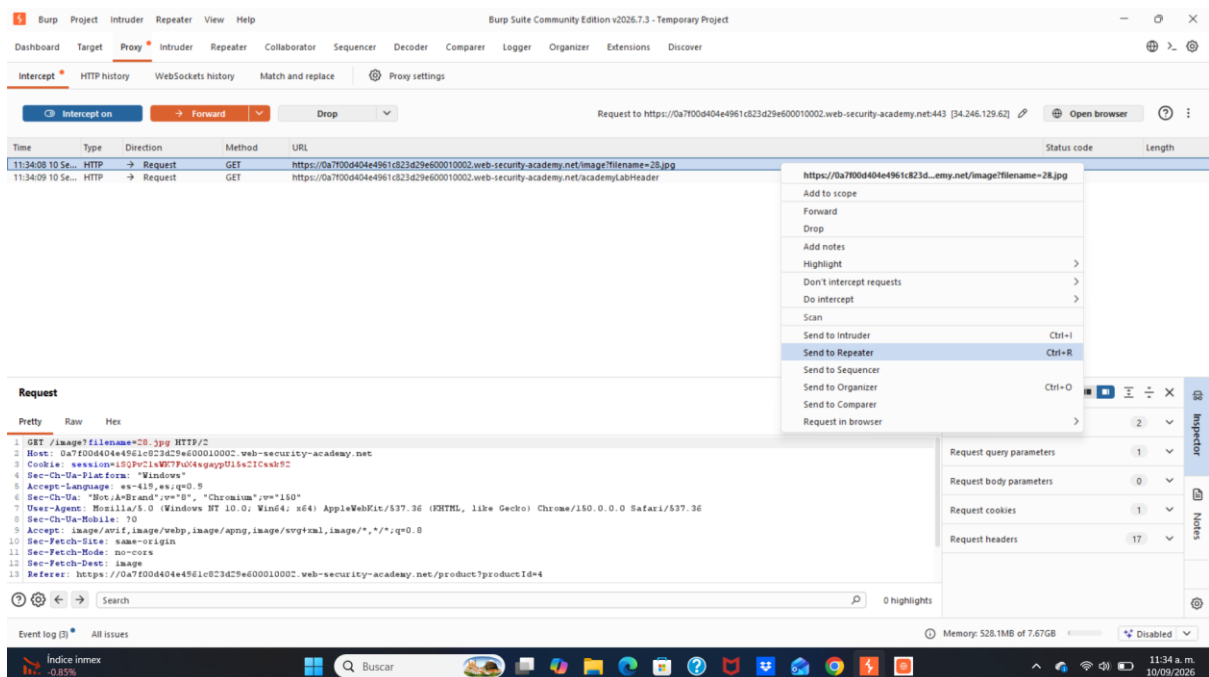
Desarrollo



Para comenzar la práctica copiamos y pegamos el link de la actividad Validation of file extension with null byte bypass en el navegador que nos proporciona burp, se nos abra la pagina web de laboratorio, daremos click en “ACCESS THE LAB” para comenzar.

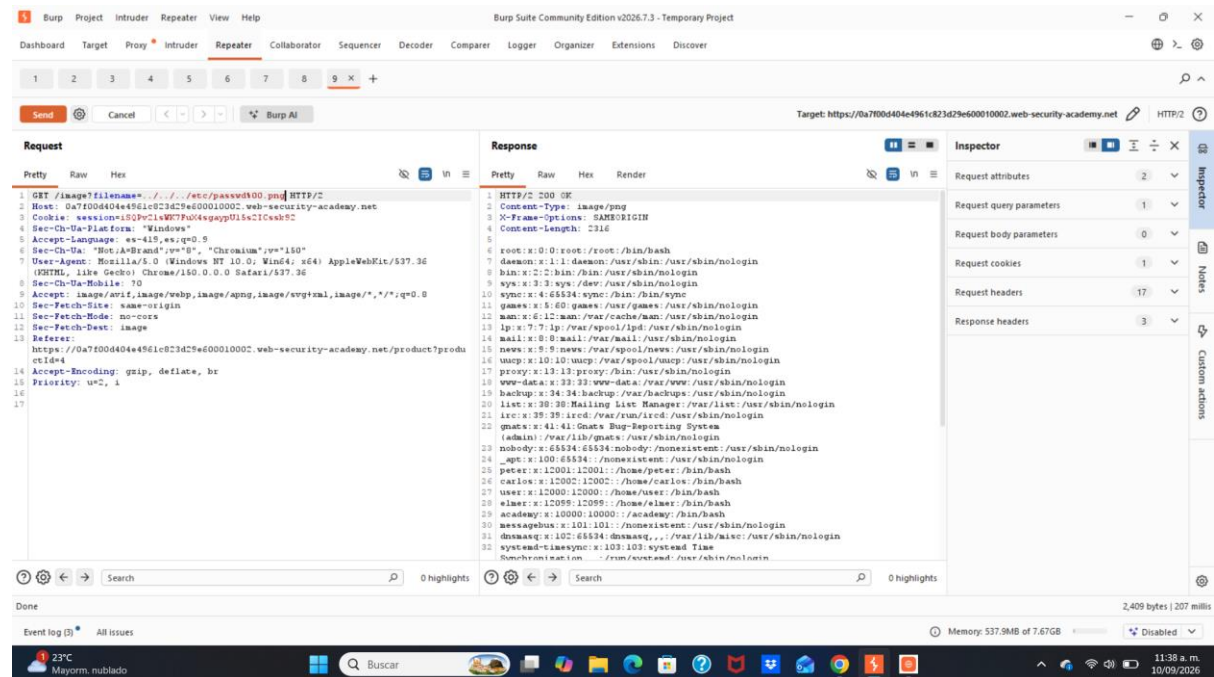


Despues de comenzar el laboratorio se nos abrirá la página web que vamos a vulnerar, en esta daremos click a cualquiera de las imágenes que nos aparezcan en el botón de “View Details”, inmediatamente nuestro burp suite comenzará a interceptar todo el tráfico proveniente de esta pagina.



Abrimos nuestro burp suite y nos daremos cuenta de que si, efectivamente nos ha empezado a llegar tráfico de la página (Todo esto en la sección de proxy), daremos click en forward para “ignorar” los logs que no nos importen, nos enfocaremos en los que nos interesen que son los de tipo img, una vez lo encontremos lo

seleccionaremos y con el click derecho lo mandaremos al repetidor donde lo analizaremos a fondo.



En la barra de navegación nos trasladaremos al “Repeater” que es donde analizaremos el log de img, en la parte de Request que se encuentra en la parte inferior izquierda notaremos que se encuentra el “filename” ahí ingresaremos “.././../etc/passwd%00.png”, esta vulnerabilidad consiste en dos procesos de verificación el primero verifica que termine en .png, como es así, pasa automáticamente la cadena al segundo proceso donde el carácter nulo “%00” finaliza la cadena y el sistema lee “.././../etc/passwd” ahí su peligrosidad no basta con solo verificar la terminación de la ruta o el inicio de la entrada, hay que impedir la entrada de cualquier carácter nulo y canonizar la ruta asegurándonos de que todo esté dentro del directorio permitido.

Conclusión

Este laboratorio demuestra que la seguridad de una aplicación no depende únicamente de que exista una validación, sino de dónde y cuándo se aplica esa validación. La inyección de byte nulo es un ejemplo perfecto de cómo dos capas del sistema pueden interpretar la misma entrada de formas completamente distintas: la aplicación valida la cadena completa y ve .png, mientras que el sistema de archivos se detiene en el %00 y accede a /etc/passwd. Esa discrepancia es la vulnerabilidad.

La lección principal es que validar una representación intermedia de la entrada no garantiza nada si el sistema subyacente usa una representación diferente. La extensión .png que la aplicación verificó nunca llegó a usarse porque el sistema operativo truncó la cadena antes. Esto refuerza un principio fundamental: la validación debe realizarse sobre la representación final y canonicalizada que se usará realmente, no sobre la entrada cruda.

Además, este caso subraya la importancia de mantener actualizado el entorno de ejecución. El null byte injection funcionaba en versiones antiguas de PHP porque las funciones del sistema de archivos aceptaban bytes nulos; las versiones modernas los rechazan automáticamente. Sin embargo, el código legado y los servidores sin actualizar siguen siendo vulnerables, lo que demuestra que la seguridad no es solo escribir código correcto, sino también gestionar el ciclo de vida del software.

Finalmente, este laboratorio nos recuerda que las defensas basadas en listas negras o validaciones superficiales siempre serán frágiles. La solución robusta combina rechazo explícito de bytes nulos, canonicalización de la ruta, verificación del destino final dentro del directorio permitido, y actualización constante del entorno. Solo así se elimina la categoría completa de vulnerabilidad, en lugar de parchear un ataque concreto.